

Introduction

Usage of digital banking has triggered a surge in sophisticated banking malware, creating a critical need for advanced security beyond traditional signature-based methods. To address this, the proposed APK-GUARD platform uses a three-tier architecture—combining edge detection, cloud-based AI, and automated response—to proactively identify and neutralize threats. Key innovations include the Honey Data Agent for tracking theft, the Time-Travel Sandbox for exposing hidden behaviour, and AI-driven malware DNA fingerprinting.

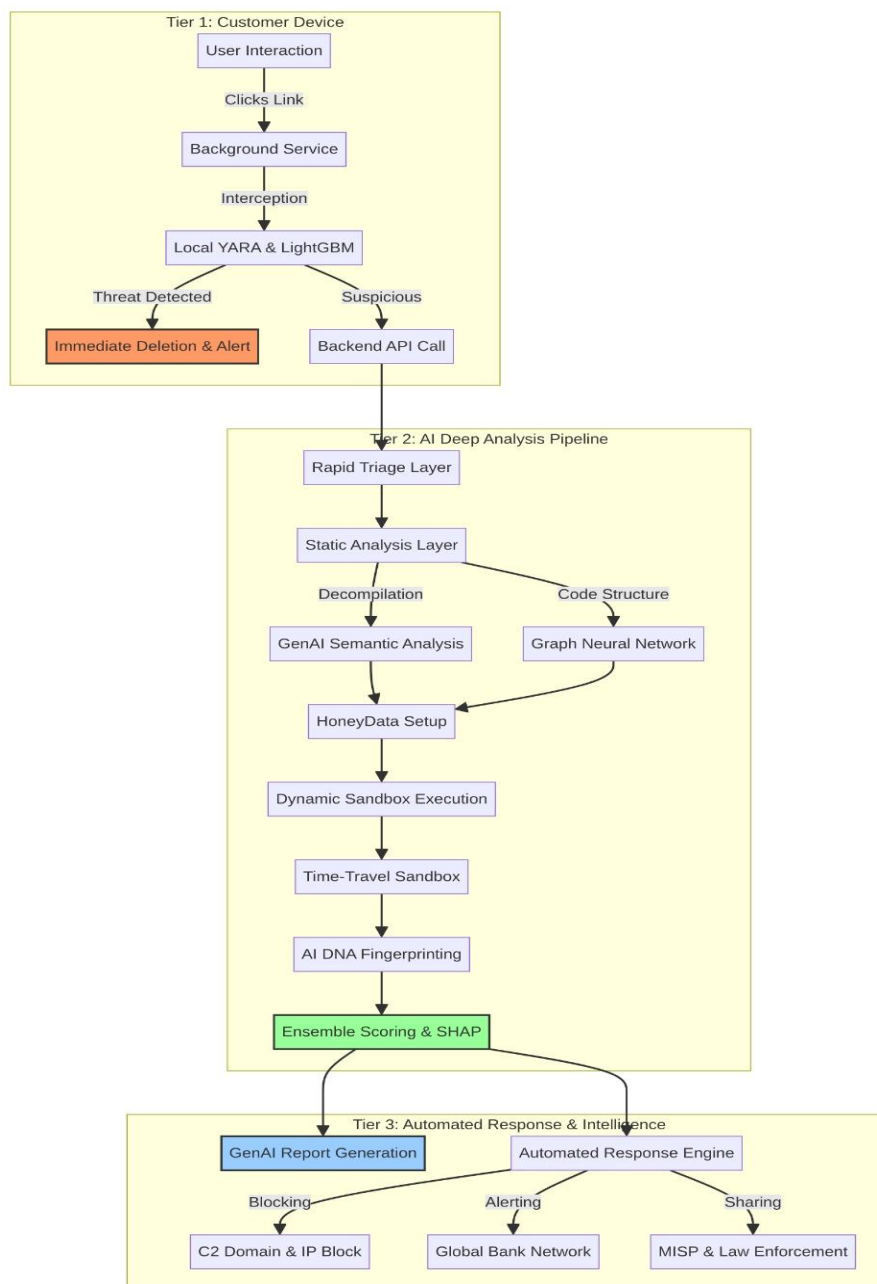
Problem Description

- Fraudsters increasingly distribute malicious mobile applications (APKs) through platforms such as WhatsApp, SMS, email, and phishing links to steal customer credentials, access sensitive information, and perform unauthorized financial transactions. Manual analysis of such APKs is complex, time-consuming, and dependent on skilled cybersecurity experts.
- The proposed solution aims to develop a ***Generative AI-powered malware analysis system*** capable of automatically analysing suspicious APK files and identifying malicious behaviour. The system should leverage GenAI for reverse engineering, malware pattern recognition, automated code interpretation, and intelligent threat summarization, along with static and dynamic analysis techniques to examine application permissions, APIs, embedded code, runtime activities, and network communications.
- Using AI-driven insights, the solution should detect malware patterns, classify threat severity, generate risk scores, and produce detailed investigation reports with actionable recommendations. The objective is to enable faster identification of fraudulent applications and support proactive cybersecurity and fraud prevention measures for bank

Proposed solution

APK-GUARD is designed as a practical, end-to-end defense system against mobile banking malware. Instead of relying on a single detection method, it combines lightweight on-device protection, deep backend analysis, and automated response. The goal is simple: stop threats early, investigate anything suspicious thoroughly, and respond fast enough to limit damage.

The system works across three connected tiers.



Tier 1: On-Device Protection

- The first layer runs directly on the customer's Android device as a small background service. It quietly monitors for newly downloaded APK files, whether they come from WhatsApp, SMS links, email attachments, or phishing websites.
- When a new APK is detected, the service performs a quick local scan using YARA rules and a trained Light model. This check takes only a couple of seconds.

- If the file is clearly malicious, installation is blocked immediately and the APK is removed. The user sees a warning explaining that a potentially harmful app was stopped.
- This approach prevents infections before they happen, without slowing down the device or interrupting normal usage.

Tier 2: Deep Backend Analysis

If an APK is unknown or shows suspicious characteristics, it is sent to the bank's backend environment for deeper inspection. This is where the system performs a more detailed, multi-step analysis.

- **Initial Screening**

The file first goes through rapid triage. Known signatures are checked, packers are identified, and entropy levels are analysed. A machine learning model assigns an early risk score to decide how aggressively the file should be examined.

- **Static Analysis**

Next, the APK is decompiled and reviewed. The system evaluates permissions, embedded libraries, API calls, and code structure. Language models and graph-based techniques help identify unusual logic patterns or hidden intent that may not be obvious through simple scanning.

- **Controlled Sandbox with Honey Data**

If the app still looks suspicious, it is executed inside an isolated Android sandbox. Before running it, the system injects carefully crafted fake data — such as dummy banking credentials or tokenized information.

If the app attempts to access or transmit this fake data, it provides clear evidence of malicious behaviour.

- **Dynamic Execution**

While running inside the emulator, the app's behaviour is monitored in real time. API calls, file access, system changes, and network traffic are logged. This helps detect actions like credential harvesting, SMS interception, overlay attacks, or communication with command-and-control servers.

- **Advanced Evasion Testing**

Some malware delays its activity or checks for emulator environments. To counter this, the system runs the APK in multiple simulated scenarios — including time changes, device reboots, and different hardware profiles. This increases the chances of exposing hidden or time-triggered behavior.

- **IOC Extraction and Fingerprinting**

If malicious activity is confirmed, the system extracts Indicators of Compromise such as IP addresses, domains, and suspicious endpoints.

It also builds a behavioural fingerprint based on hundreds of observed features. This makes it easier to identify related variants in the future, even if the code has been modified.

- **Final Risk Assessment and Reporting**

All findings are combined into a final risk score with a confidence level. The system generates clear reports tailored to both technical teams and senior management. These include a summary of what the app does, how it behaves, potential impact, and recommended next steps.

Tier 3: Automated Response

Detection alone is not enough. Once a high-risk threat is confirmed, APK-GUARD automatically initiates response actions.

This can include:

- Blocking malicious domains and IP addresses
- Updating detection rules across protected devices
- Alerting affected users
- Creating high-priority incident tickets
- Notifying security leadership

The system also shares confirmed threat intelligence with trusted banking partners through platforms like MISP. This allows other institutions to block the same infrastructure quickly, reducing the overall impact of large-scale malware campaigns.

Key Innovation Description

1. HoneyData Agent

The HoneyData Agent generates realistic but fake banking data — such as OTP messages, login credentials, contact lists, and account numbers — inside the sandbox before the suspicious APK is executed. Each data item contains a unique tracking ID.

If the malware tries to steal this data, the tracking ID appears in the captured network traffic. This gives clear proof that sensitive information is being exfiltrated and helps identify the attacker's Command and Control (C2) server

Impact:

- Provides direct evidence of data theft
- Identifies attacker infrastructure
- Reduces false positives significantly

2. Time travel sand box

An advanced sandbox that runs the APK in six parallel environments, each simulating different time and device conditions.

It fast-forwards system time, simulates reboots, mimics normal user activity, randomizes device identifiers, and installs decoy banking apps. These variations trigger malware that would otherwise stay inactive during normal testing.

Impact:

- Detects delayed and evasive malware
- Defeats anti-sandbox techniques
- Improves overall detection rates

3. AI malware DNA fingerprinting

AI Malware DNA Fingerprinting is a behavioral analysis approach that focuses on what malware does rather than how its code looks. During sandbox execution, the system extracts more than 1,000 behavioral features from the APK.

These features include API call sequences, permission usage over time, network communication patterns, and the order in which sensitive data is accessed and transmitted.

Instead of relying on code signatures, it analyzes behavior patterns such as API sequences, permission usage, and network activity. An ML model then identifies known malware families and new variants based on these behavioral “DNA” patterns.

Impact:

- Detects zero-day variants
- Works despite code obfuscation
- Reduces detection time from weeks to same-day identification

How the System Works

It operates through a meticulously designed, multi-stage workflow that ensures rapid detection and comprehensive analysis of suspicious APKs. The process is initiated either by a customer accidentally clicking a malicious link or by a bank submitting an APK for deep analysis. The system then orchestrates a series of AI-powered agents and sandboxing techniques to identify, analyse , and respond to threats.

Stage 1: On-Device Protection

- A lightweight background service runs on the customer's phone at all times.
- The moment a malicious link is tapped, the APK download begins, and the MalwareDetectionService activates immediately.
- The service extracts the SHA256 file hash and runs a local YARA scan within the first 1.5 seconds.
- It then calls the bank backend (POST /api/quickcheck) for a rapid server-side check.
- The backend queries Redis cache, a YARA signature database, and a LightGBM ML model
- If the verdict is malicious, the APK file is deleted from the device instantly.

- A warning is displayed to the user confirming the malware name, risk score, and that the file has been removed.

Stage 2: Rapid Triage

- APKs that were not immediately blocked enter the backend triage pipeline.
- The system scans against a database of over a unique known number of malware signatures using YARA.
- APKiD analysis detects whether the file has been packed or obfuscated.
- Entropy analysis checks if the code is encrypted, since unusually high entropy is a strong indicator of suspicious intent.
- A LightGBM ML model assigns an initial risk score between 0 and 40 points.
- Known malware is blocked at this stage without proceeding further.
- Unknown or suspicious APKs continue to the next layer.

Stage 3: Static Deep Analysis

- Four AI agents run in parallel, each analyzing the APK from a different angle.

Agent 1 – Decompilation

- Uses Apktool, JADX, and Androguard to convert the binary into readable Java pseudocode.
- Extracts the AndroidManifest.xml file.
- Generates call graphs showing how functions interact with each other.

Agent 2 - ML Static Scoring

- An XGBoost classifier analyzes over 100 static features including permissions, API call count, entropy, certificate details, and obfuscation indicators.
- Assigns a risk score of up to 40 points.
- Surfaces the top contributing factors behind the score.

Agent 3 - Semantic Code Analysis

- Uses CodeBERT to generate code embeddings.
- Queries a 50,000-sample malware database via ChromaDB vector retrieval.
- Runs CodeLlama-13B to interpret what the code is actually doing in plain terms, such as identifying SMS interception or credential exfiltration routines.
- Assigns a risk score of up to 30 points.

Agent 4 - Graph Neural Network

- Loads the call graph into Neo4j and runs GraphSAGE / GAT analysis.
- Detects malicious call chains, for example AccessibilityService to getText to sendSMS, which is a known SMS theft pattern.
- Assigns a risk score of up to 30 points.
- After all four agents complete, a consensus mechanism evaluates how much the agents agree with each other to determine overall confidence.

Stage 4: HoneyData Setup

A HoneyData Agent creates customized fake data based on what the static analysis found.

- For APKs suspected of OTP theft, a fake SMS is generated containing a trackable token, for example "Your SBI OTP: HONEY-OTP-ABC123".
- For credential stealers, fake usernames and passwords are created.
- For APKs targeting contacts or banking data, full fake profiles are built including names, phone numbers, account numbers, and IFSC codes.
- Every piece of fake data contains a unique embedded tracking ID.
- If the malware steals any of this data, the tracking IDs will appear in the network traffic, providing direct proof of theft.

Stage 5: Dynamic Sandbox Execution

- The APK is installed and run inside an isolated Android emulator.
- Frida is used to monitor every API call the APK makes during execution.
- tcpdump captures all outbound network traffic.

- The sandbox simulates realistic user behaviour to trigger malware activity, such as injecting fake SMS messages, displaying fake login screens, and making honeypot data accessible.
- If the malware exfiltrates a honeypot, the network capture logs the destination server address, the request structure, and the stolen data.
- This provides cryptographic proof of theft along with the attacker's C2 server details.

Stage 6: Time-Travel Sandbox

The APK is run simultaneously across six different sandbox environments to catch dormant and evasive malware.

- Run 1 - Standard: Baseline execution to catch immediate threats.
- Run 2 - Fast Forward 7 Days: System clock is advanced to trigger time-bomb payloads.
- Run 3 - Fake Reboots: Three `BOOT_COMPLETED` intents are sent to activate reboot-triggered malware.
- Run 4 - Simulated Usage: Fake app launches, received SMS messages, and browser history are injected to mimic three days of normal phone activity.
- Run 5 - Hardware Randomization: IMEI, serial number, and device ID are changed to defeat anti-sandbox detection.
- Run 6 - Custom Triggers: Fake banking apps are installed to activate malware that only targets specific applications.

If any of the six runs reveals malicious behaviour, the APK is flagged.

This approach pushes detection coverage to between 92 and 98 percent.

Stage 7: IOC Extraction

- Indicators of Compromise are extracted from all analysis data gathered so far.
- Extracted items include C2 domains, IP addresses, ports, API endpoints, and TLS certificate fingerprints.
- The IOCs are formatted into three outputs: YARA rules for endpoint detection, firewall rules for network-level blocking, and STIX bundles for threat intelligence sharing.

Stage 8: Behavioural DNA Fingerprinting

- Over 1,000 behavioural features are extracted from the sandbox execution data.
- Features include API call sequences, permission usage patterns, network traffic timing and structure, and the order in which different data types were accessed or transmitted.
- An XGBoost model uses these features to classify the malware family, for example identifying it as TeaBot with 92 percent confidence.
- This behavioural fingerprint also allows the system to detect new variants of known malware families on the same day they appear, since behaviour patterns tend to persist even when code has been rewritten.

Stage 9: Ensemble Scoring

- A meta-learner aggregates scores and signals from every previous layer.
- It produces a final risk score from 0 to 100, a confidence percentage, and a verdict such as CRITICAL MALWARE.
- SHAP values are generated to explain exactly which evidence drove the final score, for example identifying that a stolen honeypot SMS contributed 28 points and a confirmed C2 connection contributed 22 points.
- This gives analysts a transparent and auditable decision trail.

Stage 10: Report Generation

- Five generative AI agents produce different reports in parallel.
- Report 1 - Executive Summary: A two-page non-technical overview written for leadership, focused on business risk and impact.
- Report 2 - Technical Deep-Dive: A twelve-page detailed analysis for the security team covering all findings.
- Report 3 - Attack Chain Map: A stage-by-stage visual timeline of how the malware operates.
- Report 4 - Financial Impact Estimate: A calculation of the fraud amount the detection has prevented.
- Report 5 - Remediation Playbook: Step-by-step instructions for responding to and recovering from the threat.

Stage 11: Automated Response

- For a CRITICAL verdict, automated responses trigger without waiting for manual intervention.
- C2 domains are blocked via the WAF.
- C2 IP addresses are blocked via firewall rules.
- SMS alerts are sent to potentially 50,000 or more affected customers within two minutes.
- YARA detection rules are deployed to 10,000 or more enrolled customer devices.
- A Priority 1 incident ticket is created automatically.
- CISO and CEO are notified immediately.
- Enhanced monitoring is enabled on accounts linked to affected users.

Stage 12: Industry Intelligence Sharing

- A STIX bundle containing all extracted IOCs is published to MISP.
- Over 500 partner banks receive the intelligence and can auto-import the rules to update their own WAFs, typically within five minutes of publication.
- Law enforcement notifications are sent to FBI IC3, CERT-IN India, and Interpol.
- Romanian cybercrime authorities are engaged when attacker infrastructure traces back to that region.
- Domain abuse reports are filed with relevant registrars to initiate C2 server takedowns, which typically complete within 24 to 72 hours.

Technologies Used

APK-GUARD draws on tools from several different domains — Android development, cybersecurity, machine learning, cloud infrastructure, and generative AI. No single technology handles everything on its own, so the stack was assembled deliberately, with each component addressing a specific problem in the detection pipeline.

Background Service (Android Device):

- Language: Java/Kotlin
- Service Type: Foreground Service (keeps the monitor active even when the app isn't open)
- Communication: HTTPS with certificate pinning (blocks interception of traffic to the backend)
- Local Detection: YARA engine + LightGBM (on-device checks for fast initial screening, no server round-trip needed)

Backend API:

- Framework: FastAPI (Python), built for native async support when several scans run at once
- Runtime: Uvicorn (ASGI server), deployed with multiple workers
- Cache: Redis, for fast lookups and session management
- Database: PostgreSQL, the primary store for threat records, analysis reports, and system logs

Analysis Tools:

- Decompilation: Apktool, JADX-Core (convert binary APKs into readable code for inspection)
- Runtime Analysis: Frida (dynamic instrumentation that monitors live API calls and behavior)
- Network Analysis: tcpdump, Wireshark (capture and inspect traffic generated during execution)
- Signature Matching: YARA (pattern matching against known malware signatures)
- Packer Detection: APKiD (identifies obfuscation and packing techniques used to evade analysis)
- Computer Vision: OpenCV (detects fake login screens and malicious overlay UI)

Machine Learning Models:

- Triage: LightGBM, for fast initial risk assessment

- Static Scoring: XGBoost, for comprehensive static feature analysis
- Code Embeddings: CodeBERT, for semantic representations of code used in similarity comparison
- Code Generation / Semantic Analysis: CodeLlama-13B, fine-tuned for malware code interpretation and report generation
- Graph Analysis: GraphSAGE / GAT (PyTorch Geometric), for analyzing call graphs and structural patterns
- DNA Fingerprinting: XGBoost, for behavioral classification of malware families
- Ensemble: Meta-learner (scikit-learn), aggregates outputs from every model into one risk score APK-GUARD
- Explainability: SHAP (TreeExplainer), gives analysts a transparent view of why a prediction was made

Databases and Storage:

- Primary Database: PostgreSQL — threat intelligence, analysis reports, and system logs
- Caching Layer: Redis — high-speed access to frequently used data
- Vector Database: ChromaDB — stores over 50,000 malware code embeddings
- Graph Database: Neo4j — maps behavioral relationships and call graphs
- Event Streaming: Apache Kafka — real-time messaging between services
- Object Storage: S3-Compatible Storage — APK files, analysis artifacts, and generated reports

Generative AI and Large Language Models:

- Orchestration: LangChain, coordinates multi-agent workflows and manages interaction between AI components
- Supported Models: Claude-3, GPT-4, CodeLlama-13B
- Grounding: RAG architecture on ChromaDB and a custom malware knowledge base, which keeps AI output tied to real threat data instead of generic answers

Integration and Security Ecosystem:

- Incident Response: Cortex XSOAR, for automated workflow execution
- SIEM: Splunk, for centralized log and metrics management
- Threat Intelligence: MISP, for IOC sharing with partner organizations
- Cross-Validation: VirusTotal, for external verification of analysis findings
- Alerts: Firebase, for push notifications and real-time security alerts

Infrastructure and Deployment:

- Containerization: Docker, for packaging every service consistently • Orchestration: Kubernetes, for scaling and availability
- CI/CD: GitHub Actions, for automated testing and deployment
- Monitoring: Prometheus, Grafana — track performance, resource use, and service health in real time
- Hardware Requirements: NVIDIA A100 GPUs, high-core-count CPUs, large RAM, and SSD storage throughout, to support the ML workloads and LLM inference

System Architecture

APK GUARD is built using a three-tier architecture that helps detect and respond to Android banking malware efficiently. Each tier has a specific role in the detection process. The first tier works on the user's device and monitors applications for suspicious activity. The second tier uses cloud-based AI to perform a deeper analysis of the collected data and identify potential threats. The third tier handles the response process by generating alerts and taking necessary actions when malware is detected. Since each component performs a separate task, the system is easy to manage, flexible, and capable of providing faster and more accurate threat detection.

Stage	Component	Function
1	Customer Device	Source of the APK and user interactions.
2	Tier 1 Scanner	Performs initial scanning and identifies suspicious applications.
3	Backend API	Transfers data between the device and cloud analysis modules.
4	Static Layer	Examines APK code, permissions, and static features.
5	Dynamic Layer	Executes the APK in a sandbox and monitors runtime behavior.
6	DNA Engine	Uses machine learning to analyze behavior and identify malwar
7	Response Engine	Generates alerts and takes actions against detected threats.

Customer Device: The APK GUARD service runs in the background on the user's device and continuously monitors newly downloaded or installed applications.

Tier 1 Scanner:

Whenever a new APK is detected, this component performs a quick scan to check for any suspicious activity before sending it for further analysis.

Backend API: The Backend API acts as the bridge between the user's device and the analysis system, handling data transfer and coordinating the different stages of analysis.

Static Layer: This layer examines the APK file without running it. It checks the code, permissions, and other important details to identify possible security risks.

Dynamic Layer

Here, the APK is executed in a secure sandbox environment so that its behavior can be observed safely without affecting the actual device.

DNA Engine: This component analyzes the collected behavioral data and uses machine learning to identify malware families and detect new malware variants.

Response Engine: Based on the analysis results, the Response Engine determines the risk level and takes appropriate actions such as generating alerts or blocking threats.

All these components work together to provide fast and reliable protection against Android banking malware while ensuring accurate threat detection.

Example Use Case

To see how APK GUARD works, consider the following situation.

Scenario: Ankitha and a Suspicious Banking App

Ankitha, an HDFC Bank customer, receives a message on WhatsApp saying that she can get cashback by downloading a new banking application. Thinking that the message is genuine, she clicks the link and downloads a file named **hdfc-rewards.apk** on her Android phone.

APK GUARD in Action

1. Ankitha clicks the link.
2. The APK file starts downloading on her phone.
3. APK GUARD detects the downloaded file and begins scanning it.
4. The application is checked for suspicious activity.
5. The APK is sent to the backend for detailed analysis.
6. The system analyzes the application's permissions, code, and behavior.
7. The APK is identified as a malicious application and given a high risk score.
8. APK GUARD blocks the application and removes it from the device.
9. A warning message is displayed, informing Ankitha that a harmful application has been detected and blocked.

Result: The malicious application is stopped before it can access sensitive information such as banking details, passwords, or OTPs. This helps keep Ankitha's personal and financial information safe.

Expected Impact

Metric	Existing Methods	APK GUARD
Detection Accuracy	70–80%	90–95%
Detection Time	Several minutes	Few seconds
False Positives	15–20%	2–5%
Hidden Malware Detection	Low	High
New Variant Detection	Limited	Improved through AI analysis
Security Analysis Effort	High	Reduced through automation
Threat Response	Mostly Manual	Automated

Key Impact Areas

- **Improved User Protection:** Detects and blocks malicious applications before they can access sensitive banking information.
- **Reduced Financial Risk:** Helps prevent banking fraud by identifying threats at an early stage.
- **Better Malware Detection:** Combines static analysis, dynamic analysis, and machine learning to identify both known and unknown malware.
- **Time Saving:** Automates malware analysis and reporting, reducing manual effort for security teams.
- **Enhanced Security Monitoring:** Provides detailed reports that help in understanding and responding to threats more effectively.

Future Scope

- Integration of real Android devices for more accurate malware testing and analysis.

- Extension of APK GUARD to support iOS application analysis for cross platform security.
- Use of advanced AI models to improve threat detection and report generation.
- Implementation of federated learning to enhance model performance while maintaining data privacy.
- Exploration of blockchain technology for secure storage and sharing of threat information.
- Development of techniques to detect AI generated and highly sophisticated malware.
- Expansion of the platform to identify malware targeting IoT devices and connected systems.
- Addition of real time fraud monitoring and advanced behavioral analysis for stronger security protection.

Conclusion

In conclusion, APK GUARD presents a comprehensive and intelligent security framework for protecting mobile banking applications against the growing threat of banking malware. By integrating Artificial Intelligence-based analysis, dynamic sandbox testing, and behavioural malware detection techniques, the system is capable of identifying both known and emerging malicious applications with high accuracy. Innovative components such as the Honey Data Agent, Time Travel Sandbox, and AI Malware DNA Fingerprinting enhance the system's ability to detect sophisticated attack patterns and respond proactively to potential threats.

The proposed solution not only strengthens mobile banking security but also helps safeguard sensitive user information by detecting and mitigating risks at an early stage. Furthermore, its scalable and adaptive architecture ensures that it can evolve alongside the rapidly changing malware landscape. Overall, APK GUARD demonstrates an effective, reliable, and practical approach to enhancing cybersecurity in mobile banking environments, contributing to safer digital financial transactions and improved user trust.